PROGETTO FINALE DI PROGRAMMAZIONE

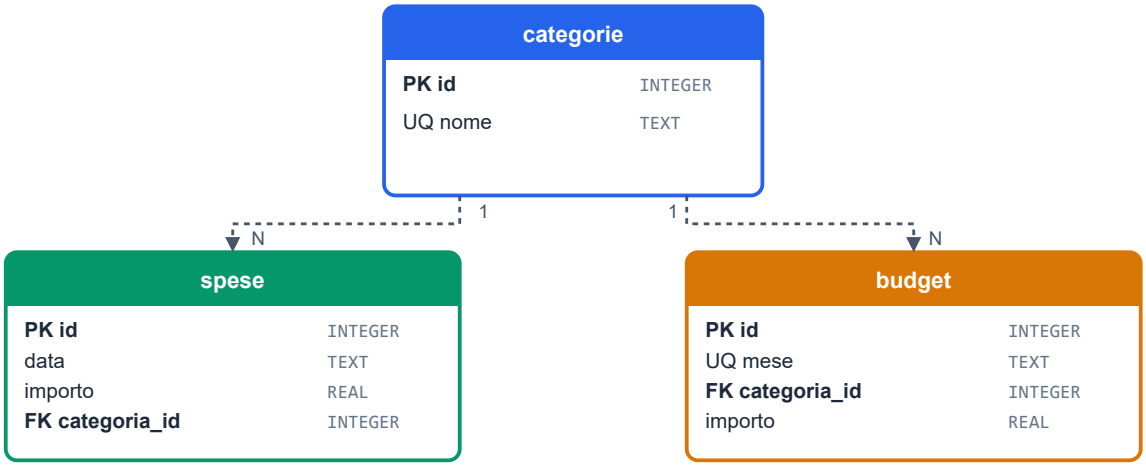
Sistema di Gestione delle Spese Personali e del Budget

1. Link al Repository GitHub

> **Repository GitHub del progetto:**
> https://github.com/Stevit/PersonalExpenseSystem.git

2. Schema Logico del Database

Lo schema relazionale si compone di tre tabelle progettate secondo le specifiche fornite, garantendo la corretta normalizzazione e l'integrità referenziale tra categorie, spese registrate e limiti di spesa mensili (budget).



Relazione	Tabella Padre	Tabella Figlia	Chiave Esterna	Cardinalità
Categorie - Spese	categorie (id)	spese (categoria_id)	FOREIGN KEY (categoria_id) REFERENCES categorie(id)	1 : N
Categorie - Budget	categorie (id)	budget (categoria_id)	FOREIGN KEY (categoria_id) REFERENCES categorie(id)	1 : N

3. Script SQL

3.1 Script di Creazione del Database (CREATE TABLE)

Di seguito è riportato integralmente lo script SQL di creazione delle tabelle, in cui compaiono esplicitamente tutti i vincoli di integrità richiesti: **PRIMARY KEY**, **FOREIGN KEY**, **CHECK**, **UNIQUE** e **NOT NULL**.

```
CREATE TABLE IF NOT EXISTS categorie (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    nome TEXT NOT NULL UNIQUE CHECK (length(nome) > 0)
);

CREATE TABLE IF NOT EXISTS spese (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    data TEXT NOT NULL CHECK (length(data) = 10),
    importo REAL NOT NULL CHECK (importo > 0),
    categoria_id INTEGER NOT NULL,
    descrizione TEXT,
    FOREIGN KEY (categoria_id) REFERENCES categorie(id)
);

CREATE TABLE IF NOT EXISTS budget (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    mese TEXT NOT NULL CHECK (length(mese) = 7),
    categoria_id INTEGER NOT NULL,
    importo REAL NOT NULL CHECK (importo > 0),
    UNIQUE (mese, categoria_id),
    FOREIGN KEY (categoria_id) REFERENCES categorie(id)
);
```

3.2 Dimostrazione Esplicita dei Vincoli di Integrità

Vincolo	Tabella	Colonna / Clausola	Scopo Applicativo
PRIMARY KEY	Tutte (categorie, spese, budget)	`id INTEGER PRIMARY KEY AUTOINCREMENT`	Identificatore univoco di ciascun record.
FOREIGN KEY	spese, budget	`FOREIGN KEY (categoria_id) REFERENCES categorie(id)`	Integrità referenziale: impedisce l'inserimento di spese o budget per categorie inesistenti.

CHECK	categorie, spese, budget	`CHECK (length(nome) > 0)`
		`CHECK (importo > 0)`
		`CHECK (length(data) = 10)`
		`CHECK (length(mese) = 7)`
			Validazione a livello DB: vieta nomi vuoti, importi minori o uguali a zero e formati non conformi.

UNIQUE	categorie, budget	`nome TEXT ... UNIQUE` `UNIQUE (mese, categoria_id)`	Evita duplicati di categorie e garantisce un solo budget per categoria in un dato mese.

NOT NULL	categorie, spese, budget	nome, data, importo, categoria_id, mese	Garantisce la presenza obbligatoria di tutti i campi essenziali per il funzionamento del sistema.

3.3 Script di Inserimento Dati di Esempio (INSERT)

Dati di esempio coerenti con lo scenario descritto e sufficienti a dimostrare sia il corretto funzionamento dei report che la rilevazione del superamento del budget.

```
INSERT INTO categorie (id, nome) VALUES (1, 'Alimentari');
INSERT INTO categorie (id, nome) VALUES (2, 'Trasporti');
INSERT INTO categorie (id, nome) VALUES (3, 'Svago');

INSERT INTO budget (id, mese, categoria_id, importo) VALUES (1, '2025-01', 1, 300.00);
INSERT INTO budget (id, mese, categoria_id, importo) VALUES (2, '2025-01', 2, 150.00);

INSERT INTO spese (id, data, importo, categoria_id, descrizione) VALUES (1, '2025-01-
10', 295.50, 1, 'Spesa supermercato');
INSERT INTO spese (id, data, importo, categoria_id, descrizione) VALUES (2, '2025-01-
15', 25.00, 1, 'Pranzo');
INSERT INTO spese (id, data, importo, categoria_id, descrizione) VALUES (3, '2025-01-
18', 120.00, 2, 'Abbonamento mezzi');
```